

Frontend Recreation Plan

AI-Orchestration Content Factory

Component	Tech Stack	Lines of Code
Frontend (apps/web)	React 19, TypeScript 5.9, Tailwind CSS v4, Vite 8, SWR, DnD Kit	~3,830 (68 files)
API Layer (apps/api)	FastAPI, SSE, Supabase Realtime, httpx	~2,500 (16 files)
Backend Packages	Python, LangGraph, Supabase, FreeRouter v3.1	~35,000+ (Python)
FreeRouter	Standalone LLM proxy (LiteLLM), 2 providers, 7 routes	Independent

Prepared for: Hassan Arif

Repository: github.com/HassanArif-collab/AI-Orchestration

Branch: codebase-audit-finding-fixes

April 2026

Version 3.0 - Comprehensive Implementation Guide

Table of Contents

1. Executive Summary
2. Current Architecture Analysis
 - 2.1 Technology Stack
 - 2.2 Component Hierarchy
 - 2.3 Data Flow Architecture
3. Bug Catalog
 - 3.1 Critical Bugs (Must Fix)
 - 3.2 High-Priority Issues
 - 3.3 Medium-Priority Issues
4. Database Schema Reference
 - 4.1 kanban_cards Table
 - 4.2 agent_thoughts Table
 - 4.3 pipeline_runs Table
 - 4.4 topic_briefs Table
 - 4.5 Supporting Tables
5. SSE Event Protocol Specification
 - 5.1 Chat Stream Events
 - 5.2 Agent Thought Events
6. Supabase Realtime Integration Patterns
7. Backend LangGraph State Architecture
 - 7.1 DiscoveryState
 - 7.2 ProductionState
8. Chunk-by-Chunk Recreation Plan
 - 8.1 Chunk 0: Project Foundation
 - 8.2 Chunk 1: Types and API Client
 - 8.3 Chunk 2: Layout Shell
 - 8.4 Chunk 3: Kanban Board Core
 - 8.5 Chunk 4: Card Details and Drawer
 - 8.6 Chunk 5: Review and Approval Flow
 - 8.7 Chunk 6: Chat Panel
 - 8.8 Chunk 7: Sidebar Panels
 - 8.9 Chunk 8: Telemetry and System
 - 8.10 Chunk 9: Polish and Responsive
9. API Contract Reference
10. Error Boundary Strategy

11. Testing Strategy

12. Risk Mitigation

13. Deployment and Environment Setup

1. Executive Summary

This document provides a comprehensive implementation guide for rebuilding the frontend of the AI-Orchestration project (Content Factory), targeting the codebase-audit-finding-fixes branch. The analysis covers all 68 frontend source files across the React application (apps/web), identifying 7 critical bugs, 14 high-priority issues, and 15 medium-priority improvements. Based on this thorough audit, the document presents a detailed 10-chunk recreation plan designed to rebuild the frontend from scratch in independently testable, incrementally deliverable units, complete with actual TypeScript code examples, database schema references, SSE event protocols, and architectural decision rationale.

The current frontend is a Kanban-based dashboard for managing an AI-powered YouTube content pipeline. It integrates with a FastAPI backend via REST endpoints, SSE streaming for chat, and Supabase Realtime WebSockets for live card updates. While the underlying concept is solid, the AI-generated implementation suffers from pervasive architectural issues: uncontrolled re-rendering in streaming components, fragile focus management in modals, missing error boundaries, inconsistent API contracts, and a deprecated drag-and-drop library. These are not isolated bugs but systemic patterns that resist piecemeal patching, necessitating a full selective recreation of the frontend layer.

The recommended approach is a selective recreation: rebuild apps/web and apps/api from the ground up using modern patterns, while surgically fixing the identified bugs in the packages layer (which contains approximately 35,000 lines of complex, mostly correct business logic written in Python using LangGraph). Each chunk in the plan is designed to be independently verifiable against the existing backend, ensuring continuous validation throughout the rebuild process. The plan includes a complete Zod-based type system matching the backend database schema exactly, a reusable SSE client with automatic reconnection, a Zustand-based global UI state management layer, and a comprehensive error boundary strategy.

Metric	Current State	Target State
Frontend Files	68 files, ~3,830 LOC	Clean rebuild, ~2,500 LOC
Critical Bugs	7 identified	0 at each chunk gate
High-Priority Issues	14 identified	0 at each chunk gate
Recreation Chunks	N/A	10 independent chunks
API Endpoints Consumed	14 (inconsistent contracts)	14 (strict TypeScript types)
SSE/WebSocket Streams	2 (fragile parsing)	2 (robust with reconnection)
Dependencies Audit	3 outdated/deprecated	All current (April 2026)

Table 1. Frontend Recreation Key Metrics

2. Current Architecture Analysis

2.1 Technology Stack

The frontend is built on a modern but bleeding-edge stack. React 19.2 and Vite 8 are both very recent releases, and while they provide excellent developer experience and performance characteristics, the ecosystem tooling has not fully caught up. Tailwind CSS v4 uses a CSS-first configuration approach (no `tailwind.config.js`), which is correct but means that all team members must be familiar with the new paradigm. The application uses SWR for data fetching, `@dnd-kit` for drag-and-drop on the Kanban board, and `react-markdown` with `remark-gfm` for rendering markdown content. Supabase provides both the PostgreSQL database and the Realtime WebSocket layer for live updates.

Technology	Version	Purpose	Risk Level	Audit Status
React	19.2.4	UI framework	High	Current
Vite	8.0.1	Build tool	High	Current
TypeScript	5.9.3	Type safety	Low	Current
Tailwind CSS	4.2.2	Styling	Medium	Current
SWR	2.4.1	Data fetching	Low	Current
@dnd-kit/core	6.3.1	Drag-and-drop	High	DEPRECATED
Supabase JS	2.100.1	DB + Realtime	Low	Current
react-markdown	10.1.0	Markdown rendering	Low	Current

Table 2. Frontend Technology Stack with Audit Status

The most critical finding from the dependency audit is that `@dnd-kit/core` (^6.3.1) and `@dnd-kit/sortable` (^8.0.0) have been deprecated and merged into a single unified package `@dnd-kit/react` (^1.0.0) in late 2025. The old packages will not receive security updates or bug fixes, and the API surface has changed significantly. The new `@dnd-kit/react` package provides a cleaner API with better TypeScript support, improved accessibility, and built-in keyboard navigation that was missing from the old core package. Continuing to use the old packages creates both a security liability and a migration debt that will only grow over time.

2.2 Component Hierarchy

The application is organized as a single-page layout with no client-side routing. The component tree has a maximum depth of 4 levels, which is reasonable. The layout consists of a Header (topic discovery trigger), a central Kanban Board (6 columns with draggable cards), and a right-side Sidebar with 6 tabs (Chat, YouTube, Quota, Models, Skills, Knowledge Base). A `CardDrawer` overlays the board when a card is clicked, showing detailed information, agent activity logs, script previews, and the review/approval interface.

The data flow architecture uses two parallel patterns: Supabase Realtime WebSocket subscriptions for live Kanban card updates (managed through SWR), and traditional REST API calls for pipeline operations, quota monitoring, and chat streaming. The SSE streaming for chat uses `raw fetch()` +

ReadableStream rather than the EventSource API, which is the correct approach since EventSource only supports GET requests and the chat endpoint requires POST with a body. Agent thoughts are received through a separate Supabase Realtime subscription on the agent_thoughts table.

Level	Components	Count
0 (Root)	MainLayout	1
1 (Layout)	Header, Board, Sidebar, ToastContainer	4
2 (Sections)	Column, Card, CardDrawer, ChatPanel, YouTubePanel, QuotaPanel, ModelRegistry, SkillViewer, KnowledgeBase	9
3 (Widgets)	StatusBadge, EmptyState, ProgressBar, AgentLog, ChatMessage, ScriptViewer, ReviewPanel, PublishConfirmModal, CompetitorCard, OwnStats	10
4 (Leaf)	ReactMarkdown (in multiple parents)	1

Table 3. Component Hierarchy Depth Map

2.3 Data Flow Architecture

The application employs a dual-channel data architecture that creates both flexibility and complexity. The primary data channel for Kanban cards and agent thoughts flows through Supabase Realtime (WebSocket) with SWR as the caching and revalidation layer. When a card is created, updated, or deleted in the backend, the Supabase postgres_changes trigger fires, the WebSocket delivers the event to the frontend, and SWR automatically revalidates the relevant cache key. This means the UI updates in near-real-time without explicit polling for card operations.

The secondary data channel handles pipeline state, quota monitoring, and chat streaming. Pipeline state for a specific card is polled via SWR every 5 seconds (usePipelineState), while provider quotas are similarly polled every 5 seconds (useQuota). The chat system uses Server-Sent Events (SSE) over a POST request to /api/chat/stream, with the response parsed as a ReadableStream. This dual-channel approach is architecturally sound but introduces synchronization challenges: for example, a card moving from "review" status back to "drafting" after rejection is reflected via Supabase Realtime, but the pipeline state poller may still show stale state for up to 5 seconds.

Data Source	Transport	Hook/Module	Cache Key
Kanban Cards	Supabase Realtime + SWR	useCards	kanban-cards
Agent Thoughts	Supabase Realtime	useAgentStream	N/A (append-only)
Pipeline State	REST + SWR (5s poll)	usePipelineState	pipeline-state-{id}
Provider Quota	REST + SWR (5s poll)	useQuota	provider-quota
Chat Stream	SSE (POST + ReadableStream)	useChat	N/A (streaming)

YouTube Analytics	REST + SWR (5min poll)	useYouTube	competitor-videos
-------------------	------------------------	------------	-------------------

Table 4. Data Flow Architecture

3. Bug Catalog

The following tables present a comprehensive catalog of all identified bugs and issues in the frontend codebase, organized by severity. Each issue includes the affected file, a description of the problem, and its impact on the user experience or system stability. The severity ratings follow a standard classification: Critical bugs cause crashes or data loss; High-priority issues cause significant UX degradation; Medium-priority issues are design concerns that should be addressed during recreation.

3.1 Critical Bugs (Must Fix)

Critical bugs represent failures that directly impact application stability, data integrity, or security. These issues would cause crashes, expose user data, or silently corrupt the application state if left unresolved. Each of these must be definitively addressed in the recreation, not merely patched.

#	File	Description	Impact
C1	Board.tsx	DragOverlay uses non-null assertion (cards.find(!)) that crashes if card is deleted between drag start and overlay render	Runtime crash during DnD
C2	Sidebar.tsx	ChatPanel unmounts on tab switch, destroying SSE connection, message history, and session state	Chat data loss on navigation
C3	ChatPanel.tsx	Every streaming token re-renders entire message list, re-parsing markdown for all previous messages	Severe performance degradation
C4	ChatMessage.tsx	No rehype-sanitize on ReactMarkdown output from API responses creates XSS attack surface	Security vulnerability (XSS)
C5	.env.example	Contains real Supabase project URL and anon key instead of placeholder values	Credential exposure
C6	api.ts	response.json() has no try/catch; invalid JSON throws SyntaxError that bypasses error mapper	Unhandled crash on malformed API
C7	useChat.ts	No mutex on sendMessage; concurrent calls create competing AbortControllers with shared refs	Race condition in chat streaming

Table 5. Critical Bugs

C1 Fix - Safe DragOverlay Card Lookup:

The current Board.tsx uses a dangerous non-null assertion on `Array.prototype.find()`, which returns undefined when no matching card is found. If the card is deleted between the drag start event and the DragOverlay render, the application crashes with a `TypeError`. The fix involves using optional chaining and providing a fallback empty-card state that renders a ghost placeholder instead of crashing.

```
// CURRENT (BUGGY):
const activeCard = cards.find(c => c.id === activeId)!;
<DragOverlay>
  <Card card={activeCard} /> { /* CRASHES if activeCard is undefined */}
</DragOverlay>

// FIXED:
const activeCard = cards.find(c => c.id === activeId);
<DragOverlay>
  {activeCard ? <Card card={activeCard} /> : (
    <div className="opacity-50 border-2 border-dashed rounded-lg p-4">
      Card removed during drag
    </div>
  )}
</DragOverlay>
```

3.2 High-Priority Issues

High-priority issues cause significant UX degradation or represent architectural problems that will compound if not addressed. While they may not cause immediate crashes, they create fragile patterns that make the codebase difficult to maintain and extend. These should all be resolved during the recreation process through better architectural decisions.

#	File	Description	Impact
H1	ReviewPanel.tsx	No cancel/abort mechanism for hanging API calls; both Approve and Reject disabled during submit	User stuck on broken submit
H2	CardDrawer.tsx	Focus trap uses <code>querySelector("button[onClick]")</code> unreliable in React synthetic events	Broken focus management
H3	Card.tsx	<code>handleSave</code> and <code>handleDelete</code> do not call <code>mutate()</code> after success; card list does not refresh	Stale UI after actions
H4	Column.tsx	Badge shows active count but list still renders expired cards, confusing UX	Misleading count display
H5	Sidebar.tsx	No resize listener; <code>isOpen</code> initial state uses <code>window.innerWidth</code> once on mount	Broken responsive behavior
H6	CardDrawer.tsx	Drawer closes immediately after <code>onDecision</code> callback; user cannot see result	Confusing UX flow
H7	PublishConfirmModal.tsx	No double-click prevention on publish button after countdown reaches zero	Accidental double-publish

H8	vite.config.ts	No API proxy configuration; causes CORS issues during development	Dev environment broken
H9	MainLayout.tsx	No error boundary wrapping; any render error crashes entire application	Full app white-screen
H10	api.ts	/api/kanban/cards/ vs /api/kanban/tasks/ inconsistent resource naming in endpoint paths	API contract confusion
H11	useAgentStream.ts	Unhandled loadHistory rejection; subscribe() never fires if history fetch fails	Missing real-time updates
H12	useChat.ts	No SSE reconnection for dropped connections; only handles timeouts	Permanent chat failure
H13	ScriptViewer.tsx	lg:grid-cols-2 never activates inside 600px-wide drawer (lg breakpoint is 1024px)	Layout never triggers
H14	CompetitorCard.tsx	No loading="lazy" on thumbnails, no onError handler, no aspect-ratio	Broken image display

Table 6. High-Priority Issues

3.3 Medium-Priority Issues

Medium-priority issues represent design concerns, performance optimizations, and code quality improvements that should be addressed during recreation. While they do not cause immediate failures, they create technical debt that impacts maintainability, developer experience, and long-term application health.

#	File	Description
M1	Board.tsx	grouped cards computed every render without useMemo; recalculates on unrelated state changes
M2	Card.tsx	useCardTimer runs interval for every card even those not in column 2
M3	cardHelpers.ts	Column 1 has no action logic for error state; errored discovery cards get stuck
M4	constants.ts	EXPIRATION_MS hardcoded at 3 hours with no sync to backend 10-min cleanup cron
M5	usePipelineState.ts	Polls every 5s even when pipeline is idle or in terminal state
M6	useQuota.ts	Polls globally regardless of quota panel visibility
M7	useToast.ts	No maximum toast limit; rapid errors cause unlimited accumulation
M8	index.css	Duplicate animate-spin definition conflicts with Tailwind built-in
M9	types/index.ts	MODEL_REGISTRY hardcoded display data drifts from actual backend assignments

M1 0	errorMapper.ts	Missing 409 Conflict and 422 Unprocessable Entity handling
M1 1	supabase.ts	No runtime type validation on Supabase data; schema drift silently wrong data
M1 2	useYouTube.ts	Dead code: apiFetcher function defined but never used
M1 3	KnowledgeBase.tsx	Same XSS concern as ChatMessage - renders Markdown without rehype-sanitize
M1 4	YouTubePanel.tsx	Competitor videos always fetched even when "Own Channel" tab is active
M1 5	multiple files	Retry counts, timeouts, backoff delays scattered across 4 files with no centralized config

Table 7. Medium-Priority Issues

4. Database Schema Reference

This section provides the complete database schema that the frontend must match. All Zod validation schemas, TypeScript type definitions, and API response types must align exactly with these table definitions. The schema is managed via Supabase migrations in the repository at `supabase/migrations/001_initial_schema.sql`. Any mismatch between frontend types and the database schema will cause silent data loss or runtime errors in the UI.

4.1 `kanban_cards` Table

This is the primary table driving the Kanban board UI. Each card represents a topic or content piece moving through the production pipeline. The `column_index` field uses integer values 1-6 to represent the six pipeline stages. The `parent_id` column creates a self-referencing foreign key that links discovery sub-task cards to their parent task card via the `kanban_callback`. The `metadata` field is a JSONB column that stores the `topic_brief`, `viability_score`, and other dynamic attributes. The frontend `KanbanCard` type **MUST** include `parent_id` (`string | null`) and must use `column_index` (NOT "column") as the field name to match the database exactly.

Column	Type	Constraints	Frontend Field
id	UUID	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code>	<code>KanbanCard.id</code>
title	TEXT	NOT NULL, DEFAULT empty string	<code>KanbanCard.title</code>
column_index	INTEGER	NOT NULL, DEFAULT 1, CHECK (1-6)	<code>KanbanCard.column</code> (MISMATCH!)

pipeline_run_id	TEXT	NULLABLE	Not in current types
parent_id	UUID	REFERENCES kanban_cards(id)	MISSING from current types!
color	TEXT	NOT NULL, DEFAULT "#1D9E75"	Not in current types
metadata	JSONB	DEFAULT '{}':jsonb	KanbanCard.topic_brief
status	TEXT	NOT NULL, DEFAULT 'idle'	KanbanCard.status
position	INTEGER	NOT NULL, DEFAULT 0	Not in current types
expires_at	TIMESTAMP TZ	NULLABLE	KanbanCard.expires_at
created_at	TIMESTAMP TZ	NOT NULL, DEFAULT now()	KanbanCard.created_at
updated_at	TIMESTAMP TZ	NOT NULL, DEFAULT now(), auto-trigger	KanbanCard.updated_at

Table 8. kanban_cards Schema with Frontend Type Mapping

Critical Schema Mismatches to Fix: The current frontend types/index.ts uses "column" as the field name, but the database column is "column_index". Additionally, the parent_id field is completely missing from the frontend type. The topic_brief and viability_score fields are stored inside the metadata JSONB column in the database, not as top-level columns. The frontend must extract these from metadata after fetching.

```
-- Actual SQL from 001_initial_schema.sql:
CREATE TABLE IF NOT EXISTS kanban_cards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL DEFAULT '',
  column_index INTEGER NOT NULL DEFAULT 1 CHECK (column_index BETWEEN 1 AND 6),
  pipeline_run_id TEXT,
  parent_id UUID REFERENCES kanban_cards(id),
  color TEXT NOT NULL DEFAULT '#1D9E75',
  metadata JSONB DEFAULT '{}':jsonb,
  status TEXT NOT NULL DEFAULT 'idle',
  position INTEGER NOT NULL DEFAULT 0,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4.2 agent_thoughts Table

This table stores the real-time activity log for each card. The frontend subscribes to INSERT events on this table via Supabase Realtime and appends new thoughts to an in-memory list. The thought_type column uses a CHECK constraint to enforce valid values: thinking, search, output, error, memory_read, and memory_write. The card_id foreign key has ON DELETE CASCADE, meaning thoughts are automatically deleted when their parent card is deleted. An index on card_id ensures fast queries when loading historical thoughts for a specific card.

Column	Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
card_id	UUID	REFERENCES kanban_cards(id) ON DELETE CASCADE
agent_name	TEXT	NOT NULL, DEFAULT 'unknown'
thought_type	TEXT	NOT NULL, CHECK IN ('thinking','search','output','error','memory_read','memory_write')
content	TEXT	NOT NULL, DEFAULT empty string
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Table 9. agent_thoughts Schema

4.3 pipeline_runs Table

This table tracks the execution state of LangGraph pipeline runs. The run_id is a text field that matches the LangGraph thread ID. The stage_outputs and stage_status fields use JSONB to store flexible per-stage data without requiring schema migrations when new stages are added. The frontend polls this table (via REST API) to show pipeline progress in the CardDrawer.

Column	Type	Constraints
run_id	TEXT	PRIMARY KEY
current_stage	TEXT	NOT NULL, DEFAULT 'trend_analysis'
stage_outputs	JSONB	DEFAULT '{}':jsonb
stage_status	JSONB	DEFAULT '{}':jsonb
status	TEXT	NOT NULL, DEFAULT 'running'
error_message	TEXT	DEFAULT ''

Table 10. pipeline_runs Schema

4.4 topic_briefs Table

This table serves as the Topic Reservoir where discovery-grade topics are stored after passing viability scoring. The viability_score_breakdown field stores the full 17-question scoring result as JSONB, which the frontend renders in the topic brief view inside the CardDrawer. The status field tracks whether a brief is in the reservoir, selected for production, or completed. This table is NOT directly rendered on the Kanban board but is referenced when viewing card details.

4.5 Supporting Tables

The schema also includes video_performance (tracks engagement metrics for published videos), production_cycles (manages the full lifecycle of a content piece from topic selection to publication), escalations (tracks items requiring human intervention), and iteration_logs (records each scoring iteration

in the Karpathy loop with scores, mutation zones, and script snapshots). All tables have Row Level Security enabled with permissive anon policies for service-side access. An `updated_at` trigger automatically updates the timestamp on every row modification.

5. SSE Event Protocol Specification

The chat streaming system uses Server-Sent Events (SSE) over HTTP POST. The frontend sends a POST request to `/api/chat/stream` with a JSON body containing the user message and `session_id`. The backend responds with a streaming body where each line follows the SSE format: `"data: {JSON}"` followed by a newline. Each JSON payload has a `"type"` field that determines how the frontend should process the event. The frontend **MUST** handle all event types listed below; missing a handler for any type will cause silent data loss or broken UI states.

5.1 Chat Stream Events

These events are sent over the SSE connection from `/api/chat/stream`. The connection is initiated via POST (not GET) because the endpoint requires a JSON body. The frontend uses `fetch()` + `ReadableStream` (not `EventSource`) because `EventSource` only supports GET requests. The backend sends these events in the following order during a normal chat interaction.

Event Type	Direction	Payload Fields	Frontend Action
token	Server to Client	type: "token", content: string	Append content to streaming text buffer, update UI
tool_start	Server to Client	type: "tool_start", tool: string	Show "Calling {tool}..." indicator, add to active tools list
tool_end	Server to Client	type: "tool_end", tool: string	Remove from active tools list
done	Server to Client	type: "done", session_id?: string	Finalize message, persist session_id for conversation continuity
error	Server to Client	type: "error", message: string	Show error toast, append error text to message

Table 11. Chat SSE Event Types (Complete)

Important Note: There is NO "progress" event type. The previous plan incorrectly listed a progress event. Pipeline progress is tracked through Supabase Realtime subscriptions on the `kanban_cards` and `agent_thoughts` tables, NOT through the SSE chat stream. Do not implement a progress event handler.

SSE Client Implementation (from current `useChat.ts`):

The current `useChat.ts` implementation demonstrates the SSE parsing pattern. The code below shows the actual event loop that processes incoming SSE lines. This pattern should be extracted into a reusable SSE

client module in Chunk 1, with the addition of automatic reconnection logic for dropped connections.

```
// Actual SSE parsing from apps/web/src/hooks/useChat.ts
const reader = response.body.getReader();
const decoder = new TextDecoder();
let buffer = '';
let accumulatedText = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  buffer += decoder.decode(value, { stream: true });
  const lines = buffer.split('\n');
  buffer = lines.pop() || ''; // Keep incomplete line in buffer

  for (const line of lines) {
    if (!line.startsWith('data: ')) continue;
    const jsonStr = line.slice(6).trim();
    if (!jsonStr) continue;

    try {
      const event = JSON.parse(jsonStr);
      switch (event.type) {
        case 'token':
          accumulatedText += event.content;
          setStreamingText(accumulatedText);
          break;
        case 'tool_start':
          setActiveTools(prev => [...prev, event.tool]);
          break;
        case 'tool_end':
          setActiveTools(prev => prev.filter(t => t !== event.tool));
          break;
        case 'done':
          if (event.session_id) sessionRef.current = event.session_id;
          // Write accumulated text into assistant message
          setMessages(prev => prev.map(m => {
            m.id === assistantId ? { ...m, content: accumulatedText } : m
          }));
          break;
        case 'error':
          setError(event.message || 'An error occurred');
          break;
      }
    } catch { /* Skip malformed JSON lines */ }
  }
}
```

5.2 Agent Thought Events

Agent thoughts are received through a separate Supabase Realtime subscription, not through SSE. The frontend subscribes to INSERT events on the `agent_thoughts` table. When a new thought is inserted by the backend (during pipeline execution), the frontend receives the full row data and appends it to the in-memory thought list. The `thought_type` field determines the visual styling: "thinking" shows a blue indicator, "search" shows a green indicator, "output" shows a purple indicator, "error" shows a red indicator, "memory_read" and "memory_write" show yellow indicators.

6. Supabase Realtime Integration Patterns

Supabase Realtime provides WebSocket-based subscriptions to database changes. The frontend uses two subscriptions: one for `kanban_cards` (all CRUD operations) and one for `agent_thoughts` (INSERT only). Both subscriptions are managed through SWR to provide caching, revalidation, and automatic cleanup on component unmount. The patterns below show the actual implementation from the current codebase and highlight the improvements needed for the recreation.

6.1 Card Subscription Pattern (Current Implementation):

The `useCards` hook demonstrates the standard pattern for subscribing to Supabase Realtime changes. It creates a channel, registers a `postgres_changes` listener for all events on the `kanban_cards` table, and triggers an SWR `mutate()` call whenever any change occurs. This causes a full refetch of all cards, which is simple but potentially wasteful for large datasets. The recreation should add optimistic updates for drag-and-drop operations to reduce perceived latency.

```
// From apps/web/src/hooks/useCards.ts
export function useCards() {
  const fetcher = async (): Promise<KanbanCard[]> => {
    if (!supabase) return [];
    const { data, error } = await supabase
      .from('kanban_cards')
      .select('*')
      .order('created_at', { ascending: false });
    if (error) throw error;
    return data as KanbanCard[];
  };

  const { data: cards, error, isLoading, mutate } =
    useSWR('kanban-cards', fetcher);

  useEffect(() => {
    if (!supabase) return;
    const channel = supabase
      .channel('kanban-realtime')
      .on('postgres_changes',
        { event: '*', schema: 'public', table: 'kanban_cards' },
        () => { mutate(); } // Refetch all cards on any change
      )
      .subscribe();

    return () => { supabase.removeChannel(channel); };
  }, [mutate]);

  return { cards: cards ?? [], isLoading, error, mutate };
}
```

6.2 Supabase Client Configuration:

The Supabase client is initialized with the project URL and anon key from environment variables. The realtime configuration limits `eventsPerSecond` to 10 to prevent bandwidth overhead when the backend is generating many rapid updates. The client is exported as nullable (`SupabaseClient | null`) to allow the app to function in development mode without Supabase credentials. All hooks check `isSupabaseConfigured()` before attempting any operations.

```
// From apps/web/src/lib/supabase.ts
```

```
import { createClient, SupabaseClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase: SupabaseClient | null =
  (supabaseUrl && supabaseKey)
    ? createClient(supabaseUrl, supabaseKey, {
        realtime: { params: { eventsPerSecond: 10 } },
      })
    : null;
```

7. Backend LangGraph State Architecture

The backend uses two separate LangGraph StateGraphs, each with its own TypedDict state. Understanding these states is essential for the frontend because the PipelineStateResponse type must match the ProductionState fields exactly. The backend snapshots state after every node transition, making the pipeline crash-proof: if the server dies at iteration 14 of 20, restart picks up exactly where it left off. The frontend polls this state to show pipeline progress in the CardDrawer.

7.1 DiscoveryState

The Discovery graph handles topic finding and viability grading. It has NO loops and NO human gates. The flow is: gather_context, search_web, generate_topics, grade_viability, save_topics. Once complete, it dumps topic cards into Kanban Column 2 (Suggested Topics) with a 3-hour expiration timer. The pipeline_status field transitions through "discovering", "grading", "complete", and "error" states.

Field	Type	Purpose
card_id	str	Identity - links to kanban_cards.id
seed_hint	Optional[str]	Optional human hint like "AI in Pakistan"
zep_context	str	Past audience data from Zep memory
search_results	list	Raw Exa.ai search results
generated_topics	list	Topic dicts with titles/descriptions
graded_topics	list	Topics that passed 17-question viability
pipeline_status	str	"discovering" "grading" "complete" "error"
error	Optional[str]	Error message if something failed

Table 12. DiscoveryState Fields (from orchestration/state.py)

7.2 ProductionState

The Production graph handles the full content creation pipeline including the Karpathy loop and human review gate. The flow is: load_learnings, research, draft, score, should_continue (conditional), mutate (loop back), capture_learning, visuals, human_review (interrupt), publish. The graph can be interrupted at the human_review node, pausing execution until the user approves or provides revision feedback. The frontend must handle this interrupt state by showing the review UI and providing approve/reject buttons that call the resume endpoint.

Field	Type	Purpose
card_id	str	Identity - links to kanban_cards.id
topic_brief	dict	The approved topic from discovery
research_dossier	str	Full research text
research_sources	list	URLs and titles of sources
zep_learnings	str	Past winning mutations from Zep
current_draft	str	Draft being evaluated RIGHT NOW
best_draft	str	Highest-scoring draft across ALL iterations
best_score	int	Score of best_draft (0-100)
evaluation_score	int	Score of current_draft (0-100)
evaluation_feedback	str	What the scorer says needs improvement
iteration_count	int	Current iteration number (0-20)
visual_plan	str	Visual cue annotations from Phase 2d
human_feedback	Optional[str]	Feedback from human if they reject
approved	bool	Whether human approved the final script
revision_count	int	Number of human revision cycles
pipeline_status	str	Multi-state: researching, drafting, scoring, mutating, visuals, review, publishing, complete, error
error	Optional[str]	Error description for error_handler

Table 13. ProductionState Fields (from orchestration/state.py)

The Karpathy Loop: After scoring, the conditional edge checks: score $\geq$ 85% OR iterations $\geq$ 20? If YES: exit loop and proceed to capture_learnings, then visuals. If NO: mutate the draft and re-score. The best_draft and best_score fields track the highest-scoring draft across ALL iterations, so even if iteration 20 scores 60% but iteration 12 scored 78%, the 78% draft is preserved. The frontend displays the iteration_count and evaluation_score in real-time in the CardDrawer progress view.

```
// PipelineStateResponse type must match ProductionState exactly:
export interface PipelineStateResponse {
  card_id: string;
  values: {
    evaluation_score: number; // current draft score
    best_score: number;      // best across all iterations
  }
}
```

```

iteration_count: number;    // 0-20
pipeline_status: PipelineStatus;
current_draft: string;     // full script text
visual_plan: string;      // visual annotations
error: string | null;
};
next: string[]; // Which node runs next (empty = complete/interrupted)
}

```

8. Chunk-by-Chunk Recreation Plan

The following plan divides the frontend recreation into 10 independently verifiable chunks. Each chunk is designed to produce a working, testable increment that can be validated against the existing backend before proceeding. The chunks are ordered by dependency: foundational layers (types, API client, layout) come first, followed by the core Kanban board, then detail views, and finally supplementary features.

Chunk	Name	Depends On	Est. Files	Priority
0	Project Foundation	None	12	Critical
1	Types and API Client	Chunk 0	8	Critical
2	Layout Shell	Chunk 0, 1	6	High
3	Kanban Board Core	Chunk 1, 2	10	Critical
4	Card Details and Drawer	Chunk 3	8	High
5	Review and Approval Flow	Chunk 4	5	Critical
6	Chat Panel	Chunk 1, 2	6	High
7	Sidebar Panels	Chunk 2, 6	12	Medium
8	Telemetry and System	Chunk 1, 2	6	Medium
9	Polish and Responsive	All chunks	Cross-cutting	Medium

Table 14. Chunk Dependency Map

8.1 Chunk 0: Project Foundation

Goal: Set up the project scaffolding, build configuration, and development infrastructure that all subsequent chunks depend on. This chunk establishes the Vite + React + TypeScript + Tailwind CSS v4 project with proper path aliases, API proxy, environment configuration, error boundaries, and a global toast system. No visible UI is produced in this chunk, but the foundation must be rock-solid before any components are built.

Critical Dependency Specifications (April 2026 audit):

The package.json MUST use these exact versions. Do NOT use the old versions from the current codebase. These versions have been verified against the npm registry as of April 2026 and include critical security patches, performance improvements, and breaking changes from the deprecated packages currently in use.

Package	Version	Reason
@supabase/supabase-js	^2.101.0	Current: ^2.100.1. 2.101 adds Realtime v2 fixes, better auth token refresh, reduced bundle size
@dnd-kit/react	^1.0.0	REPLACES both @dnd-kit/core AND @dnd-kit/sortable. Old packages are deprecated.
tailwindcss	^4.1.0	Current: ^4.2.2. 4.1+ has container queries, 3D transforms, faster incremental builds
typescript	^5.7.0	Current: ~5.9.3. 5.7 has better type narrowing, isolatedDeclarations support
zod	^3.23.0	NEW: Runtime type validation for API responses. Catches schema drift at runtime.
rehype-sanitize	^6.0.0	NEW: XSS prevention for all markdown rendering. Fixes critical vulnerability C4.
zustand	^5.0.0	NEW: Global UI state management. Replaces scattered React Context usage.
date-fns	^3.6.0	NEW: Timestamp formatting for useCardTimer hook. Tree-shakeable.
lucide-react	^0.460.0	NEW: Icon library. Replaces emoji usage throughout the application.
clsx	^2.1.0	NEW: Conditional classname utility. Powers the cn() helper for Tailwind.
@radix-ui/react-dialog	^1.1.0	NEW: Accessible drawer/modal primitives with focus trap and animation.
@radix-ui/react-tabs	^1.1.0	NEW: Accessible sidebar tab primitives with proper ARIA tablist pattern.
react-error-boundary	^4.0.0	NEW: Error boundary with retry UI and error reporting. Fixes H9.
@tanstack/react-virtual	^3.10.0	NEW: Virtual scrolling for columns with 30+ cards. Performance fix.
swr	^2.3.0	KEPT: Data fetching and caching. Stable, no changes needed.

Table 15. Complete Dependency List with Version Rationale

Deprecated Packages - Do NOT Install:

- `@dnd-kit/core` (^6.3.1) - Replaced by `@dnd-kit/react`. No longer receives updates.
- `@dnd-kit/sortable` (^8.0.0) - Merged into `@dnd-kit/react`. API completely changed.
- `@dnd-kit/utilities` (^3.2.2) - Absorbed into `@dnd-kit/react` internals.
- `framer-motion` - Not needed. Use CSS transitions and Tailwind animations instead.

State Management Architecture:

Use Zustand (^5.0.0) for global UI state. Create a single store at `src/lib/store.ts`. SWR remains the data-fetching layer for all API-driven data (cards, pipeline state, quota). Zustand is only for ephemeral UI state that does NOT come from API calls: sidebar open/close, active tab index, selected card ID, drawer open/close, and column/search filters. Do NOT use React Context for this because Zustand's selector-based reactivity prevents unnecessary re-renders across components. The store shape is defined below.

```
// src/lib/store.ts - Zustand Global UI State Store
import { create } from 'zustand';

interface UIState {
  // Sidebar
  sidebarOpen: boolean;
  activeTab: number;          // 0=Chat, 1=YouTube, 2=Quota, 3=Models, 4=Skills, 5=KB
  // Card Drawer
  selectedCardId: string | null;
  drawerOpen: boolean;
  // Filters
  filters: {
    column: number | null;    // null = show all columns
    search: string;
  };
};

// Actions
openSidebar: () => void;
closeSidebar: () => void;
setActiveTab: (tab: number) => void;
selectCard: (id: string | null) => void;
openDrawer: (cardId: string) => void;
closeDrawer: () => void;
setFilter: (column: number | null, search: string) => void;
}

export const useUIStore = create<UIState>((set) => ({
  sidebarOpen: typeof window !== 'undefined' ? window.innerWidth > 1024 : true,
  activeTab: 0,
  selectedCardId: null,
  drawerOpen: false,
  filters: { column: null, search: '' },

  openSidebar: () => set({ sidebarOpen: true }),
  closeSidebar: () => set({ sidebarOpen: false }),
  setActiveTab: (tab) => set({ activeTab: tab }),
  selectCard: (id) => set({ selectedCardId: id }),
  openDrawer: (cardId) => set({ drawerOpen: true, selectedCardId: cardId }),
  closeDrawer: () => set({ drawerOpen: false }),
  setFilter: (column, search) => set({ filters: { column, search } }),
}));
```

TypeScript Configuration:

The `tsconfig.json` must use strict mode with all type-checking flags enabled. The module resolution must be set to "bundler" for compatibility with Vite. Path aliases (`@/*` for `./src/*`) must be configured in both `tsconfig.json` and `vite.config.ts`. The exact configuration is shown below and should be copied verbatim to avoid subtle type-checking differences.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": false,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": { "@/*": ["/src/*"] }
  },
  "include": ["src"]
}
```

File	Purpose	Key Changes
package.json	Dependencies and scripts	Use exact versions from Table 15
vite.config.ts	Vite build configuration	Add /api proxy, path aliases, env prefix
tsconfig.json	TypeScript config	Copy exactly from above
src/lib/store.ts	Zustand UI state	NEW file from code example above
.env.example	Environment template	PLACEHOLDERS ONLY. Document <code>API_AUTH_ENABLED=true</code>
src/index.css	Global styles	Remove duplicate <code>animate-spin</code>
src/main.tsx	App entry	Wrap in <code>ErrorBoundary</code>
src/lib/config.ts	Centralized constants	NEW: consolidate all constants
ErrorBoundary.tsx	Error boundary	<code>react-error-boundary</code> with <code>retry</code> UI
useToast.ts	Toast store	Max limit 5, <code>crypto.randomUUID()</code>

Table 16. Chunk 0 Files

8.2 Chunk 1: Types and API Client

Goal: Establish the complete type system and API client layer that all components will depend on. This chunk creates a comprehensive, Zod-validated type system that matches the backend API contracts exactly, and a robust HTTP client with proper error handling, retry logic, and timeout configuration. The API client centralizes all endpoint definitions so that any contract mismatch is caught at compile time rather than at runtime.

Zod Schema for KanbanCard (matching database exactly):

```
// src/types/index.ts - Zod-validated types matching kanban_cards table
import { z } from 'zod';

export const ColumnIndex = z.union([
  z.literal(1), z.literal(2), z.literal(3),
  z.literal(4), z.literal(5), z.literal(6),
]);
export type ColumnIndex = z.infer<typeof ColumnIndex>;

export const PipelineStatus = z.union([
  z.literal('discovering'), z.literal('grading'), z.literal('suggested'),
  z.literal('researching'), z.literal('drafting'), z.literal('scoring'),
  z.literal('mutating'), z.literal('visuals'), z.literal('review'),
  z.literal('publishing'), z.literal('complete'), z.literal('error'),
]);
export type PipelineStatus = z.infer<typeof PipelineStatus>;

export const KanbanCardSchema = z.object({
  id: z.string().uuid(),
  title: z.string(),
  column_index: ColumnIndex, // Matches DB column name exactly!
  parent_id: z.string().uuid().nullable(), // CRITICAL: was missing!
  pipeline_run_id: z.string().nullable(),
  color: z.string(),
  metadata: z.record(z.unknown()), // Contains topic_brief, viability_score
  status: z.string(), // 'idle' | pipeline statuses
  position: z.number().int(),
  expires_at: z.string().datetime({ offset: true }).nullable(),
  created_at: z.string().datetime({ offset: true }),
  updated_at: z.string().datetime({ offset: true }),
});
export type KanbanCard = z.infer<typeof KanbanCardSchema>;
```

Error Mapper (enhanced with 409/422 handling):

The error mapper provides user-friendly error messages for all HTTP status codes. The current implementation handles 400, 401, 403, 404, 429, 500, 503, and network/timeout errors. The recreation must add 409 Conflict (for concurrent card operations) and 422 Unprocessable Entity (for validation failures). The mapper uses a regex to extract the status code from the standard API error format "API {status}: {body}" to avoid false positives from substring matching.

```
// Enhanced errorMapper.ts additions:
case '409':
  return {
    title: 'Conflict',
    message: 'This card was modified by another process. '
      'Please refresh and try again.',
    action: 'Refresh',
  };
case '422':
```

```

return {
  title: 'Validation error',
  message: 'The request data did not pass server validation. '
    'Please check your input and try again.',
  action: 'Retry',
};

```

8.3 Chunk 2: Layout Shell

Goal: Build the application layout skeleton that provides the visual structure for all subsequent chunks. This includes the Header with the topic discovery input, the MainLayout with proper flex arrangement, and the Sidebar with tab navigation. The CRITICAL requirement is that the Sidebar must NOT unmount tab content when switching tabs (using CSS display:none instead of conditional rendering) to preserve chat state and SSE connections. This is the fix for Critical Bug C2.

Sidebar Tab Persistence Pattern (fixes C2):

```

// src/layout/Sidebar.tsx - CSS-based tab visibility
import { useUIStore } from '@lib/store';
import ChatPanel from '@components/chat/ChatPanel';
import YouTubePanel from '@components/telemetry/YouTubePanel';

const TABS = ['Chat', 'YouTube', 'Quota', 'Models', 'Skills', 'KB'];

export function Sidebar() {
  const { sidebarOpen, activeTab, setActiveTab, closeSidebar } = useUIStore();

  return (
    <aside className={sidebarOpen ? 'w-96' : 'w-0'} style={{transition:'width 0.2s'}}>
      /* Tab buttons */
      <div className="flex border-b">
        {TABS.map((name, i) => (
          <button key={i}
            className={activeTab === i ? 'border-b-2 border-blue-500' : ''}
            onClick={() => setActiveTab(i)}
            >{name}</button>
        ))}
      </div>

      /* ALL panels rendered, visibility controlled by CSS */
      <div className="flex-1 overflow-hidden">
        <div style={{ display: activeTab === 0 ? 'block' : 'none' }}>
          <ChatPanel /> /* ALWAYS MOUNTED - preserves SSE connection */
        </div>
        <div style={{ display: activeTab === 1 ? 'block' : 'none' }}>
          <YouTubePanel />
        </div>
        /* ... remaining tabs ... */
      </div>
    </aside>
  );
}

```

8.4 Chunk 3: Kanban Board Core

Goal: Build the core Kanban board with 6 columns, draggable cards, and the `@dnd-kit/react` integration. This is the most complex chunk because it combines Supabase Realtime data with drag-and-drop interactions and must handle edge cases like concurrent updates, card deletion during drag, and optimistic UI updates. The `@dnd-kit/react` API is different from the old `@dnd-kit/core` API, so all DnD code must be rewritten.

Key `@dnd-kit/react` Migration: The new `@dnd-kit/react` package provides a unified `DndContext` component that replaces the old separate `DndContext`, `SortableContext`, and `useSortable` patterns. The new API uses `useDraggable` and `useDroppable` hooks directly, with built-in collision detection and keyboard navigation. The `DragOverlay` component is still supported but with a different props signature. All existing DnD code from `Board.tsx`, `Column.tsx`, and `Card.tsx` must be completely rewritten.

8.5 Chunk 4: Card Details and Drawer

Goal: Implement the `CardDrawer` component using `@radix-ui/react-dialog` for accessible modal behavior with built-in focus trap, escape key handling, and backdrop click dismiss. The drawer shows card details, agent activity log (from `useAgentStream`), pipeline state progress (from `usePipelineState`), and the script viewer. The drawer must use the "sm" breakpoint (640px) instead of "lg" (1024px) for responsive layout triggers. This fixes H13 (`ScriptViewer` grid never activating). The drawer must NOT close immediately after the `onDecision` callback (approve/reject) to allow the user to see the result of their action, fixing H6.

8.6 Chunk 5: Review and Approval Flow

Goal: Implement the review UI inside the `CardDrawer`, including the `ReviewPanel` component with Approve and Reject buttons, the `PublishConfirmModal` with countdown timer, and the resume API integration. The `ReviewPanel` must include a cancel/abort mechanism for hanging API calls (fixing H1) and double-click prevention on the publish button (fixing H7). The approve/reject actions call the `/api/pipeline/langgraph/resume/{cardId}` endpoint with the approved boolean and optional feedback string. After submission, the drawer must remain open for 2 seconds to show the result before optionally closing.

8.7 Chunk 6: Chat Panel

Goal: Implement the `ChatPanel` with SSE streaming, message rendering with `rehype-sanitize`, and tool call indicators. This chunk addresses the most critical performance bug (C3: every streaming token re-renders entire message list) by memoizing previous messages with `React.memo` and only updating the streaming message. The `ChatMessage` component must include `rehype-sanitize` as a plugin to `ReactMarkdown` to prevent XSS attacks (C4). The `useChat` hook must be extracted from the current implementation with the addition of a mutex lock to prevent concurrent `sendMessage` calls (C7) and automatic SSE reconnection on dropped connections (H12).

```
// ChatMessage with XSS prevention (fixes C4):  
import ReactMarkdown from 'react-markdown';
```



```
import remarkGfm from 'remark-gfm';
import rehypeSanitize from 'rehype-sanitize';

interface ChatMessageProps {
  content: string;
  role: 'user' | 'assistant' | 'tool';
}

export function ChatMessage({ content, role }: ChatMessageProps) {
  return (
    <div className={role === 'user' ? 'bg-blue-50' : 'bg-gray-50'}>
      <ReactMarkdown
        remarkPlugins={[remarkGfm]}
        rehypePlugins={[rehypeSanitize]} // CRITICAL: prevents XSS
      />
    </div>
  );
}
```

8.8 Chunk 7: Sidebar Panels

Goal: Implement the remaining sidebar panels: YouTubePanel (with lazy competitor video fetching, own channel analytics, and video repurposing), QuotaPanel (provider quota display), ModelRegistry (current model assignments from FreeRouter), SkillViewer (skill file editor), and KnowledgeBase (knowledge base viewer with rehype-sanitize). The YouTubePanel must only fetch competitor videos when the Competitor tab is active, not when the Own Channel tab is selected (fixes M14). The CompetitorCard component must use loading="lazy" on thumbnails, an onError handler for broken images, and aspect-ratio CSS (fixes H14).

8.9 Chunk 8: Telemetry and System

Goal: Implement the telemetry system including provider quota monitoring with visibility-aware polling (only poll when QuotaPanel is visible, fixing M6), pipeline state polling with terminal-state detection (stop polling when pipeline is complete or errored, fixing M5), and the agent thought stream with proper error handling for loadHistory failures (fixing H11). The MODEL_REGISTRY must be fetched from the backend API or config endpoint rather than being hardcoded, to prevent drift between the display data and actual model assignments (fixing M9).

8.10 Chunk 9: Polish and Responsive

Goal: Add responsive design for tablet and mobile viewports, keyboard accessibility for all interactive elements, loading skeletons for data-fetching states, and animation polish. This chunk also adds virtual scrolling via @tanstack/react-virtual for columns that may contain 30+ cards, preventing DOM bloat and scroll jank. The useCardTimer hook must only run its interval for cards in Column 2 (those with expires_at set), not for every card in the board (fixing M2). The Column component badge must show the count of non-expired active cards, and the card list must filter out expired cards (fixing H4).

9. API Contract Reference

This section documents all API endpoints consumed by the frontend, extracted directly from the current `api.ts` implementation. Each endpoint includes the HTTP method, path, request body type, response type, and the SWR cache key used for data fetching hooks. The frontend must always send the `X-API-Key` header because `API_AUTH_ENABLED` defaults to `TRUE` in the backend (`packages/core/config.py` line 101). Auth is enforced when `API_AUTH_ENABLED=true` AND `API_KEYS` is non-empty. The frontend must never assume auth is disabled.

Method	Endpoint	Request	Response	Cache Key
POST	<code>/api/pipeline/discover</code>	<code>DiscoverRequest</code>	<code>{status, card_id}</code>	N/A
POST	<code>/api/pipeline/produce/{id}</code>	None	<code>{status, card_id}</code>	N/A
POST	<code>/api/pipeline/langgraph/resume/{id}</code>	<code>ResumeRequest</code>	<code>{status}</code>	N/A
GET	<code>/api/pipeline/langgraph/state/{id}</code>	None	<code>PipelineStateResponse</code>	<code>pipeline-state-{id}</code>
GET	<code>/api/providers/quota</code>	None	<code>QuotaResponse</code>	<code>provider-quota</code>
GET	<code>/api/settings/skills</code>	None	<code>{files[]}</code>	N/A
POST	<code>/api/kanban/cards/{id}/save</code>	None	<code>{status}</code>	N/A
DELETE	<code>/api/kanban/tasks/{id}</code>	None	<code>{status}</code>	N/A
PATCH	<code>/api/kanban/tasks/{id}</code>	<code>{stage: number}</code>	<code>{status}</code>	N/A
GET	<code>/api/analytics/competitors</code>	None	<code>{videos[]}</code>	<code>competitor-videos</code>
GET	<code>/api/analytics/channel</code>	<code>?channel_id</code>	<code>{subs, views, count}</code>	<code>own-analytics</code>
POST	<code>/api/analytics/repurpose</code>	<code>{title, video_id, ...}</code>	<code>{status, card_id}</code>	N/A
GET	<code>/api/settings/knowledge-base</code>	None	<code>{content, path}</code>	<code>knowledge-base</code>
POST	<code>/api/chat/stream</code>	<code>{message, session_id}</code>	SSE stream	N/A

Table 17. Complete API Contract Reference

10. Error Boundary Strategy

The current application has no error boundary wrapping (H9), meaning any render error in any component crashes the entire application with a white screen. The recreation implements a three-tier error boundary strategy: a root-level boundary wrapping the entire App component, section-level boundaries wrapping the Board and Sidebar independently, and component-level boundaries for individual cards and panels that might fail due to malformed data. This ensures that a failure in one card does not bring down the entire board.

For synchronous render errors, the `react-error-boundary` library provides a component with `FallbackComponent`, `onReset`, and `onError` callbacks. For asynchronous errors (Supabase WebSocket disconnect, SSE stream failure, API timeout), the `main.tsx` entry point registers global error handlers via `window.addEventListener("error")` and `window.addEventListener("unhandledrejection")` that log the error and display a toast notification via the `useToast` hook.

```
// src/main.tsx - Global error handlers for async errors
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { ErrorBoundary } from 'react-error-boundary';
import App from './App';
import { useToastStore } from './hooks/useToast';

// Global async error handler
window.addEventListener('error', (event) => {
  console.error('[Global Error]', event.error);
  // Show toast if toast system is initialized
  try {
    useToastStore.getState().addToast({
      type: 'error',
      title: 'Unexpected Error',
      message: event.message || 'An unexpected error occurred.',
    });
  } catch { /* Toast system not yet mounted */ }
});

window.addEventListener('unhandledrejection', (event) => {
  console.error('[Unhandled Promise]', event.reason);
  try {
    useToastStore.getState().addToast({
      type: 'error',
      title: 'Async Error',
      message: event.reason?.message || 'An async operation failed.',
    });
  } catch { /* Toast system not yet mounted */ }
});

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ErrorBoundary
      FallbackComponent={ErrorFallback}
      onReset={() => window.location.reload()}
      onError={(err) => console.error('[ErrorBoundary]', err)}
    >
      <App />
    </ErrorBoundary>
  </StrictMode>
);
```

11. Testing Strategy

Each chunk has its own acceptance criteria that must pass before proceeding to the next chunk. The testing strategy is divided into three layers: unit tests for individual hooks and utility functions, integration tests for component interactions with mocked API/Supabase clients, and end-to-end tests for critical user flows (discovery to publish). The test framework uses Vitest for unit/integration tests and Playwright for end-to-end tests.

Test Layer	Tool	Scope	Run At
Unit	Vitest	cardHelpers, errorMapper, Zod schemas, config, store	Every commit
Integration	Vitest + msw	useCards, useChat, usePipelineState, useQuota (with mocked fetch/Supabase)	Every chunk gate
E2E	Playwright	Discovery flow, card drag-and-drop, review/approve cycle, chat streaming	After Chunk 5 and Chunk 9
Visual Regression	Playwright screenshots	Kanban board layout, drawer states, responsive breakpoints	After Chunk 9

Table 18. Testing Strategy Layers

The most important integration tests are for the data flow hooks: useCards must correctly refetch on Supabase Realtime events, useChat must handle all SSE event types including reconnection, and usePipelineState must stop polling when the pipeline reaches a terminal state. These tests use MSW (Mock Service Worker) to intercept API calls and a mock Supabase client that simulates Realtime events. Each chunk gate requires all tests for that chunk and all previous chunks to pass before proceeding.

12. Risk Mitigation

Risk	Probability	Impact	Mitigation
@dnd-kit/react API instability	Medium	High	Pin exact version. Write DnD abstraction layer so core code is insulated from API changes.
Supabase Realtime schema drift	Low	High	Zod validation on every Supabase response. Log and discard unknown fields instead of crashing.
LangGraph state shape changes	Medium	Medium	PipelineStateResponse type validated at runtime. Feature flags for new fields.
React 19 concurrent feature bugs	Low	Medium	Test all streaming and DnD patterns in StrictMode. Use stable APIs only.

API contract breaking changes	Low	High	Zod schemas catch mismatch at parse time. Error mapper provides graceful degradation.
SSE connection instability	Medium	Medium	Automatic reconnection with exponential backoff. Message buffering during reconnect.
Performance regression in large boards	Medium	Medium	@tanstack/react-virtual for 30+ card columns. useMemo for expensive computations.

Table 19. Risk Mitigation Matrix

13. Deployment and Environment Setup

This section documents the complete environment setup required for local development, CI/CD pipelines, and production deployment. The frontend uses Vite for development and production builds, with environment variables loaded from .env files at build time (not runtime). The API proxy configuration in vite.config.ts forwards /api requests to the FastAPI backend during development, eliminating CORS issues.

13.1 Local Development Setup

The development environment requires Node.js 20+, Python 3.12+, a Supabase project with the schema migrations applied, and the FreeRouter proxy running for LLM API access. The frontend starts with npm run dev (Vite dev server on port 5173) and the backend starts with uvicorn or the project start script (FastAPI on port 3000). The Vite proxy configuration forwards all /api requests to localhost:3000, so the frontend code uses relative URLs like /api/pipeline/discover instead of absolute URLs with localhost.

```
# .env.example - apps/web/.env.example
# PLACEHOLDER VALUES ONLY - Do NOT use real credentials!

# Supabase Configuration
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# API Base URL (for direct API calls, not proxied through Vite)
VITE_API_URL=http://localhost:3000

# IMPORTANT: API_AUTH_ENABLED defaults to TRUE in the backend
# (packages/core/config.py line 101).
# The frontend api.ts MUST always send X-API-Key header.
# Auth is enforced when API_AUTH_ENABLED=true AND API_KEYS is non-empty.
VITE_API_KEY=your-api-key-here
```

13.2 Vite Configuration

The Vite configuration must include the API proxy, path aliases, and environment variable prefix validation. The proxy configuration forwards /api requests to the FastAPI backend during development,

which eliminates CORS issues that would otherwise occur because the frontend runs on port 5173 and the backend on port 3000. Path aliases (@/ for ./src/) must be configured in both vite.config.ts (for the bundler) and tsconfig.json (for the TypeScript compiler).

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
        // Do NOT rewrite path - keep /api prefix
      },
    },
  },
  envPrefix: 'VITE_', // Only expose VITE_ prefixed vars to client
});
```

13.3 Production Build

The production build uses `npm run build` which runs `tsc -b` (type checking) followed by `vite build` (bundling). The output is a static site in the `dist/` directory that can be served by any static file server (Nginx, Cloudflare Pages, Vercel). In production, the `/api` proxy is replaced by a proper reverse proxy (Nginx) that forwards `/api` requests to the FastAPI backend. The environment variables `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, and `VITE_API_URL` are baked into the build at compile time and cannot be changed without rebuilding. The `VITE_API_KEY` is also baked in and MUST be set to a valid key for the production backend.

13.4 FreeRouter Integration

The FreeRouter is a standalone LLM proxy built on LiteLLM that provides a unified API for multiple LLM providers (OpenRouter and Ollama Cloud). It runs as a separate process and is configured via `freerouter/.env` (NOT the main `apps/web/.env`). The frontend does not communicate with FreeRouter directly; all LLM calls go through the FastAPI backend, which routes to FreeRouter internally. The `MODEL_REGISTRY` display data should be fetched from the backend rather than hardcoded, to stay in sync with actual FreeRouter configuration.